

Proyecto de Empresa Aliada: Segmentación Comercial con Clustering

Análisis de Datos de Ventas para la Marca Vanish

En este proyecto se desarrolla un análisis de segmentación utilizando técnicas de **Machine Learning No Supervisado**, específicamente el algoritmo **K-Means Clustering**, con el objetivo de identificar patrones y segmentos relevantes dentro de los datos de ventas asociados a la marca Vanish.

El análisis de clustering permite agrupar productos o regiones que presentan comportamientos similares en términos de ventas, características de producto o desempeño comercial. Este tipo de metodología es ampliamente utilizada en analítica empresarial para comprender mejor el mercado y apoyar la toma de decisiones estratégicas.

A través de este análisis se busca identificar segmentos clave que permitan detectar oportunidades de mejora en la estrategia comercial de la marca.

Objetivos del proyecto

Los principales objetivos de este análisis son:

- Explorar y comprender la estructura de los datos disponibles.
- Seleccionar variables relevantes para el proceso de segmentación.
- Aplicar técnicas de estandarización de datos.
- Implementar el algoritmo de **K-Means** para identificar grupos o clusters.
- Determinar el número óptimo de clusters utilizando el **método del codo**.
- Visualizar los resultados del clustering mediante gráficos.
- Analizar los clusters obtenidos para identificar **insights comerciales relevantes**.

Flujo de trabajo del análisis

El desarrollo del proyecto seguirá las siguientes etapas:

1. Carga y exploración inicial de los datos
2. Preparación y limpieza de los datasets
3. Selección de variables para clustering
4. Estandarización de características
5. Aplicación del algoritmo K-Means
6. Determinación del número óptimo de clusters
7. Visualización de los segmentos identificados
8. Interpretación de resultados e insights de negocio

Este tipo de análisis permite comprender cómo se agrupan los productos o regiones según su comportamiento de ventas, facilitando la identificación de áreas donde la marca puede fortalecer su presencia o mejorar su estrategia comercial.

```
In [1]: # =====  
# Importación de Librerías  
# =====  
  
import pandas as pd  
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# Configuración visual
sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (10,6)
```

Carga de los datasets

En esta sección se cargan los diferentes archivos que conforman el modelo de datos del proyecto.

Cada archivo contiene información relevante sobre distintos aspectos del negocio.

Los datasets incluidos son:

- **DIM_CALENDAR**: información temporal utilizada para análisis de ventas por fecha.
- **DIM_PRODUCT**: características y atributos de los productos.
- **DIM_SEGMENT**: segmentación de mercado o clasificación estratégica.
- **FACT_SALES**: tabla principal que contiene los registros de ventas.

La integración de estos datasets permitirá construir una vista analítica que será utilizada posteriormente para aplicar técnicas de clustering.

```
In [2]: # =====
# Carga de datasets
# =====

calendar = pd.read_excel("DIM_CALENDAR (2).xlsx")

product = pd.read_excel("DIM_PRODUCT (1).xlsx")

segment = pd.read_excel("DIM_SEGMENT (1).xlsx")

sales = pd.read_csv("FACT_SALES (1).csv")

# Mostrar primeras filas de cada dataset
print("Calendar")
display(calendar.head())

print("Product")
display(product.head())

print("Segment")
display(segment.head())

print("Sales")
display(sales.head())
```

Calendar

	WEEK	YEAR	MONTH	WEEK_NUMBER	DATE
0	01-21	2021	1	1	2021-01-10
1	02-21	2021	1	2	2021-01-17
2	03-21	2021	1	3	2021-01-24
3	04-21	2021	1	4	2021-01-31
4	05-21	2021	2	5	2021-02-07

Product

	MANUFACTURER	BRAND	ITEM	ITEM_DESCRIPTION	CATEGORY	FORMAT	A
0	INDS. ALEN	CLORALEX	0000075000592	CLORALEX EL RENDIDOR BOT.PLAST. 250ML NAL. 000...	1	LIQUIDO	C
1	INDS. ALEN	CLORALEX	0000075000608	CLORALEX EL RENDIDOR BOT.PLAST. 500ML NAL. 000...	1	LIQUIDO	C
2	INDS. ALEN	CLORALEX	0000075000615	CLORALEX EL RENDIDOR BOT.PLAST. 950ML NAL. 000...	1	LIQUIDO	C
3	INDS. ALEN	CLORALEX	0000075000622	CLORALEX EL RENDIDOR BOT.PLAST. 2000ML NAL 000...	1	LIQUIDO	C
4	INDS. ALEN	CLORALEX	0000075000639	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000...	1	LIQUIDO	C

Segment

	CATEGORY	ATTR1	ATTR2	ATTR3	FORMAT	SEGMENT
0	1	COLORO	COLORO	BAMBINO	LIQUIDO	BLEACH
1	1	COLORO	COLORO	GERMICIDA	LIQUIDO	BLEACH
2	1	COLORO	COLORO	MASCOTAS	LIQUIDO	BLEACH
3	1	COLORO	COLORO	MULTIUSOS	GEL	BLEACH
4	1	COLORO	COLORO	MULTIUSOS	LIQUIDO	BLEACH

Sales

	WEEK	ITEM_CODE	TOTAL_UNIT_SALES	TOTAL_VALUE_SALES	TOTAL_UNIT_AVG_WEE
0	34-22	7501058792808BP2	0.006	0.139	
1	34-22	7501058715883	0.487	116.519	
2	34-22	7702626213774	1.391	68.453	
3	34-22	7501058716422	0.022	1.481	
4	34-22	7501058784353	2.037	182.839	

Exploración inicial de los datos

Antes de aplicar cualquier modelo de Machine Learning es fundamental comprender la estructura de los datos disponibles.

En esta etapa se revisan aspectos clave como:

- Número de registros
- Tipos de datos
- Variables disponibles
- Posibles valores faltantes

Este análisis exploratorio permite identificar qué variables serán útiles para el proceso de segmentación mediante clustering.

```
In [3]: # =====  
# Información de Los datasets  
# =====  
  
print("Calendar Dataset")  
calendar.info()  
  
print("\nProduct Dataset")  
product.info()  
  
print("\nSegment Dataset")  
segment.info()  
  
print("\nSales Dataset")  
sales.info()
```

Calendar Dataset
<class 'pandas.DataFrame'>
RangeIndex: 156 entries, 0 to 155
Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	WEEK	156 non-null	str
1	YEAR	156 non-null	int64
2	MONTH	156 non-null	int64
3	WEEK_NUMBER	156 non-null	int64
4	DATE	156 non-null	datetime64[us]

dtypes: datetime64[us](1), int64(3), str(1)
memory usage: 6.2 KB

Product Dataset
<class 'pandas.DataFrame'>
RangeIndex: 505 entries, 0 to 504
Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	MANUFACTURER	505 non-null	str
1	BRAND	505 non-null	str
2	ITEM	503 non-null	str
3	ITEM_DESCRIPTION	505 non-null	str
4	CATEGORY	505 non-null	int64
5	FORMAT	505 non-null	str
6	ATTR1	499 non-null	str
7	ATTR2	505 non-null	str
8	ATTR3	499 non-null	str

dtypes: int64(1), str(8)
memory usage: 35.6 KB

Segment Dataset
<class 'pandas.DataFrame'>
RangeIndex: 53 entries, 0 to 52
Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	CATEGORY	53 non-null	int64
1	ATTR1	53 non-null	str
2	ATTR2	53 non-null	str
3	ATTR3	52 non-null	str
4	FORMAT	53 non-null	str
5	SEGMENT	53 non-null	str

dtypes: int64(1), str(5)
memory usage: 2.6 KB

Sales Dataset
<class 'pandas.DataFrame'>
RangeIndex: 122002 entries, 0 to 122001
Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	WEEK	122002 non-null	str
1	ITEM_CODE	122002 non-null	str
2	TOTAL_UNIT_SALES	122002 non-null	float64
3	TOTAL_VALUE_SALES	122002 non-null	float64
4	TOTAL_UNIT_AVG_WEEKLY_SALES	122002 non-null	float64
5	REGION	122002 non-null	str

dtypes: float64(3), str(3)
memory usage: 5.6 MB

```
In [4]: # =====  
# Revisar columnas de cada dataset  
# =====  
  
print("Calendar Columns")  
print(calendar.columns)  
  
print("\nProduct Columns")  
print(product.columns)  
  
print("\nSegment Columns")
```

```
print(segment.columns)

print("\nSales Columns")
print(sales.columns)
```

Calendar Columns

```
Index(['WEEK', 'YEAR', 'MONTH', 'WEEK_NUMBER', 'DATE'], dtype='str')
```

Product Columns

```
Index(['MANUFACTURER', 'BRAND', 'ITEM', 'ITEM_DESCRIPTION', 'CATEGORY',
      'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3'],
      dtype='str')
```

Segment Columns

```
Index(['CATEGORY', 'ATTR1', 'ATTR2', 'ATTR3', 'FORMAT', 'SEGMENT'], dtype='str')
```

Sales Columns

```
Index(['WEEK', 'ITEM_CODE', 'TOTAL_UNIT_SALES', 'TOTAL_VALUE_SALES',
      'TOTAL_UNIT_AVG_WEEKLY_SALES', 'REGION'],
      dtype='str')
```

Comprensión del modelo de datos

Antes de construir el dataset analítico es necesario comprender cómo se relacionan los distintos datasets disponibles.

El conjunto de datos sigue una estructura común en analítica empresarial conocida como **modelo estrella**, donde:

- La tabla **FACT_SALES** contiene las métricas principales del negocio (ventas).
- Las tablas **DIM_PRODUCT**, **DIM_SEGMENT** y **DIM_CALENDAR** contienen información descriptiva o dimensional que permite contextualizar dichas ventas.

Este modelo facilita la integración de datos para realizar análisis avanzados, como segmentación mediante algoritmos de clustering.

```
In [5]: # =====
# Tamaño de Los datasets
# =====

print("Calendar shape:", calendar.shape)
print("Product shape:", product.shape)
print("Segment shape:", segment.shape)
print("Sales shape:", sales.shape)
```

Calendar shape: (156, 5)
Product shape: (505, 9)
Segment shape: (53, 6)
Sales shape: (122002, 6)

```
In [6]: # =====
# Construcción del dataset analítico de ventas
# =====

sales_summary = sales.groupby("ITEM_CODE").agg({

    "TOTAL_UNIT_SALES": "sum",
    "TOTAL_VALUE_SALES": "sum",
    "TOTAL_UNIT_AVG_WEEKLY_SALES": "mean"

}).reset_index()

# Renombrar columnas para facilitar análisis
sales_summary.columns = [
    "item_code",
    "total_units",
    "total_revenue",
    "avg_weekly_units"
]
```

```
sales_summary.head()
```

Out[6]:

	item_code	total_units	total_revenue	avg_weekly_units
0	0000075000592	6.042	24.663	4.860489
1	0000075000608	1423.786	12423.156	35.166597
2	0000075000615	46461.839	658584.568	92.527904
3	0000075000622	48445.151	1139233.787	112.098525
4	0000075000639	28334.417	1146849.632	60.226977

Construcción del dataset analítico

Para realizar el análisis de clustering es necesario construir un dataset que represente el comportamiento de ventas de los productos.

En esta etapa se agregan las ventas utilizando la variable **ITEM_CODE**, generando métricas clave como:

- Unidades totales vendidas
- Ingresos totales generados
- Promedio de ventas semanales

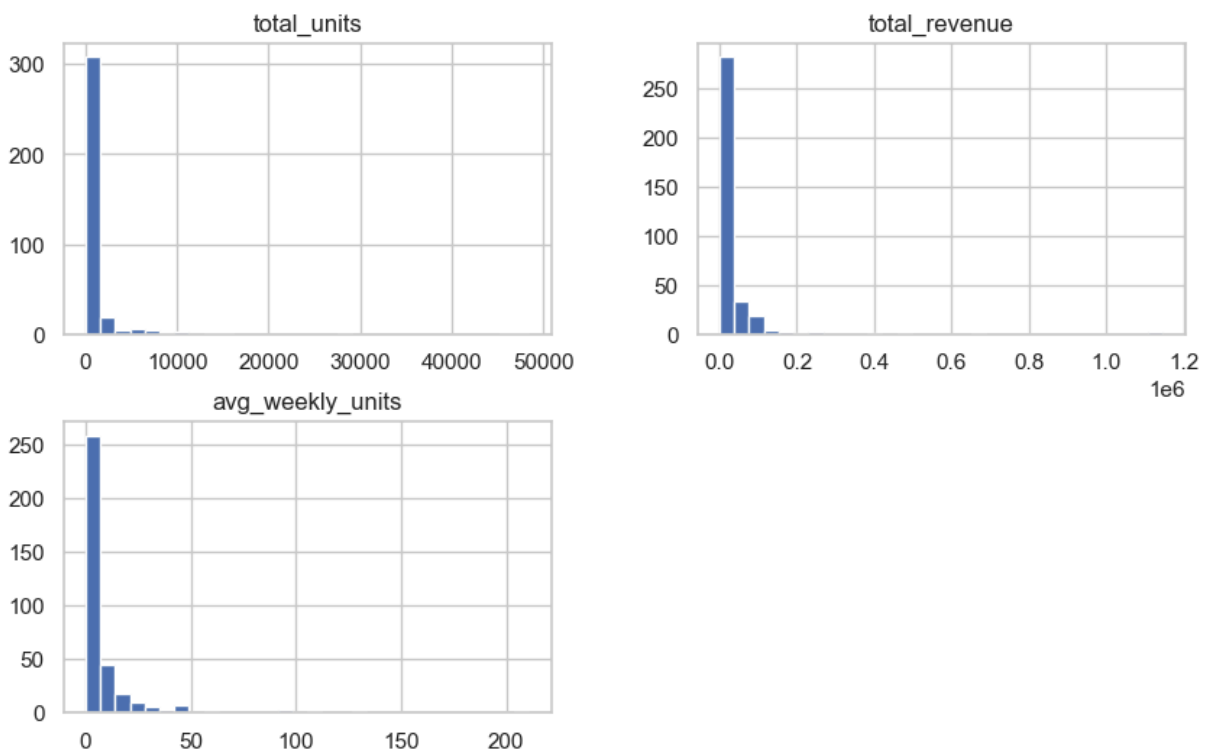
Estas métricas permiten capturar el desempeño comercial de cada producto y facilitan la aplicación de técnicas de segmentación mediante algoritmos de clustering.

```
In [7]: # =====  
# Selección de variables para clustering  
# =====  
  
features = sales_summary[[  
  
    "total_units",  
    "total_revenue",  
    "avg_weekly_units"  
  
]]  
  
features.head()
```

Out[7]:

	total_units	total_revenue	avg_weekly_units
0	6.042	24.663	4.860489
1	1423.786	12423.156	35.166597
2	46461.839	658584.568	92.527904
3	48445.151	1139233.787	112.098525
4	28334.417	1146849.632	60.226977

```
In [8]: # =====  
# Distribución de variables  
# =====  
  
features.hist(bins=30)  
  
plt.show()
```



Estandarización de las variables

Antes de aplicar el algoritmo K-Means es necesario estandarizar las variables seleccionadas. Esto es importante porque K-Means calcula distancias entre los puntos de datos, por lo que las variables con escalas mayores podrían influir más en el resultado del modelo.

Para evitar este problema se utiliza **StandardScaler**, una técnica que transforma los datos para que todas las variables tengan una media cercana a 0 y una desviación estándar de 1.

Este proceso garantiza que todas las variables tengan la misma importancia dentro del algoritmo de clustering.

```
In [9]: # =====  
# Estandarización de variables  
# =====  
  
scaler = StandardScaler()  
  
scaled_features = scaler.fit_transform(features)  
  
scaled_features[:5]
```

```
Out[9]: array([[ -0.26014222, -0.30365632, -0.19716224],  
               [ 0.07114678, -0.18423625,  1.51622772],  
               [10.59533981,  6.03947507,  4.75921374],  
               [11.05878699, 10.66900251,  5.86566085],  
               [ 6.35944426, 10.74235697,  2.93304449]])
```

Determinación del número óptimo de clusters

Para aplicar el algoritmo K-Means es necesario determinar cuántos clusters se deben crear.

Una técnica común para identificar el número óptimo de clusters es el **método del codo (Elbow Method)**.

Este método consiste en ejecutar el algoritmo K-Means con diferentes valores de K y calcular la suma de las distancias de los puntos a su centroide correspondiente.

El punto donde la reducción de la distancia comienza a disminuir de forma más lenta indica el número adecuado de clusters.

```
In [10]: # =====  
# Método del Codo
```



```
# =====

inertia = []

k_range = range(1,10)

for k in k_range:

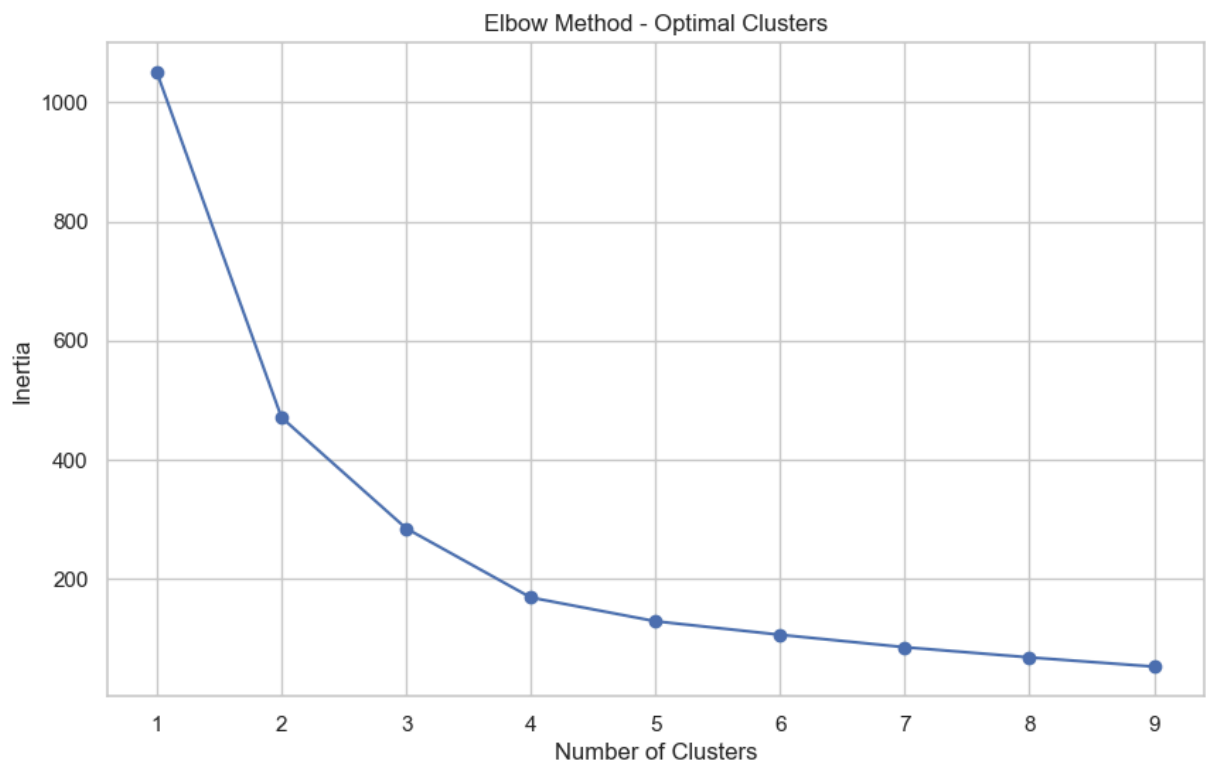
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=20)
    kmeans.fit(scaled_features)

    inertia.append(kmeans.inertia_)

plt.plot(k_range, inertia, marker='o')

plt.title("Elbow Method - Optimal Clusters")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")

plt.show()
```



Aplicación del algoritmo K-Means

Una vez identificado el número óptimo de clusters mediante el método del codo, se aplica el algoritmo K-Means para segmentar los productos en diferentes grupos.

Cada cluster representará un conjunto de productos que comparten características similares en términos de comportamiento de ventas.

```
In [11]: # =====
# Aplicar K-Means
# =====

kmeans = KMeans(n_clusters=3, random_state=42, n_init=20)

clusters = kmeans.fit_predict(scaled_features)

sales_summary["cluster"] = clusters

sales_summary.head()
```

Out[11]:

	item_code	total_units	total_revenue	avg_weekly_units	cluster
0	0000075000592	6.042	24.663	4.860489	0
1	0000075000608	1423.786	12423.156	35.166597	2
2	0000075000615	46461.839	658584.568	92.527904	1
3	0000075000622	48445.151	1139233.787	112.098525	1
4	0000075000639	28334.417	1146849.632	60.226977	1

In [12]:

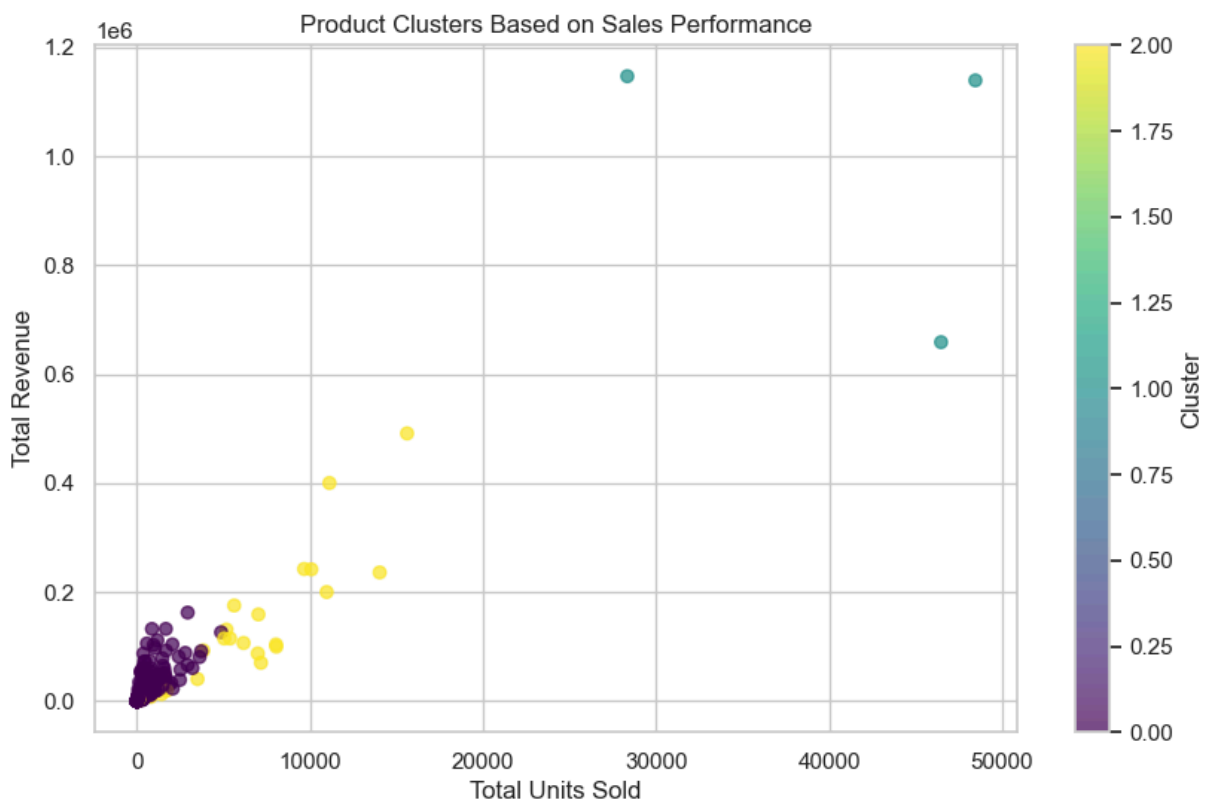
```
# =====  
# Centroides del modelo KMeans  
# =====  
  
centroids = pd.DataFrame(  
    scaler.inverse_transform(kmeans.cluster_centers_),  
    columns=features.columns  
)  
  
centroids["cluster"] = range(len(centroids))  
  
print("Centroides de los clusters:")  
display(centroids)
```

Centroides de los clusters:

	total_units	total_revenue	avg_weekly_units	cluster
0	360.591279	15127.516865	4.517443	0
1	41080.469000	981555.995667	88.284468	1
2	5481.792964	116875.500929	43.422564	2

In [13]:

```
plt.figure(figsize=(10,6))  
  
scatter = plt.scatter(  
    sales_summary["total_units"],  
    sales_summary["total_revenue"],  
    c=sales_summary["cluster"],  
    cmap="viridis",  
    alpha=0.7  
)  
  
plt.xlabel("Total Units Sold")  
plt.ylabel("Total Revenue")  
plt.title("Product Clusters Based on Sales Performance")  
  
plt.colorbar(scatter, label="Cluster")  
  
plt.show()
```



Visualización de clusters mediante PCA

Para visualizar mejor los clusters generados por el algoritmo K-Means, se utiliza la técnica de **Análisis de Componentes Principales (PCA)**.

PCA permite reducir la dimensionalidad de los datos transformando las variables originales en un número menor de componentes que conservan la mayor parte de la información del dataset.

En este caso, se reducen las variables a **dos componentes principales**, lo que permite representar gráficamente los clusters en un plano bidimensional.

```
In [14]: # =====
# Reducción de dimensionalidad con PCA
# =====

pca = PCA(n_components=2)

pca_features = pca.fit_transform(scaled_features)

pca_df = pd.DataFrame(
    pca_features,
    columns=["PCA1", "PCA2"]
)

pca_df["cluster"] = clusters

pca_df.head()
```

Out[14]:

	PCA1	PCA2	cluster
0	-0.443879	0.025117	0
1	0.672262	1.371321	2
2	12.617824	-1.358944	1
3	16.273644	-2.368220	1
4	11.956254	-3.571885	1

```
In [15]: plt.figure(figsize=(10,6))

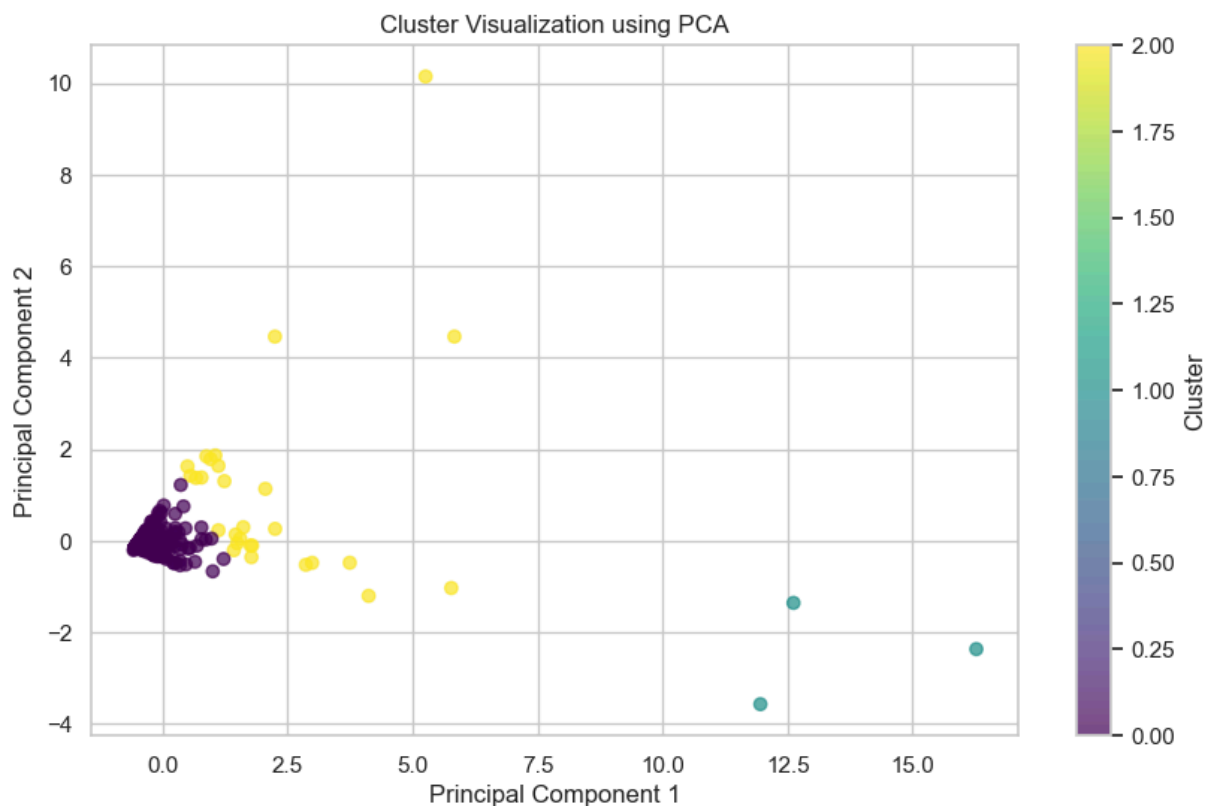
scatter = plt.scatter(
```

```
pca_df["PCA1"],
pca_df["PCA2"],
c=pca_df["cluster"],
cmap="viridis",
alpha=0.7
)

plt.title("Cluster Visualization using PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.colorbar(scatter, label="Cluster")

plt.show()
```



Interpretación de los clusters

Después de aplicar el algoritmo K-Means se identificaron diferentes grupos de productos que comparten características similares en términos de desempeño de ventas.

Cada cluster representa un segmento de productos con patrones de comportamiento específicos.

El análisis de estos segmentos permite identificar:

- Productos con **alto volumen de ventas**
- Productos con **ventas moderadas**
- Productos con **bajo desempeño comercial**

Este tipo de segmentación permite a las empresas identificar oportunidades estratégicas, como fortalecer la presencia de productos exitosos o revisar estrategias comerciales para productos con bajo rendimiento.

```
In [16]: # =====
# Análisis de clusters (solo variables numéricas)
# =====

sales_summary.groupby("cluster").mean(numeric_only=True)
```

Out[16]:

	total_units	total_revenue	avg_weekly_units
cluster			
0	360.591279	15127.516865	4.517443
1	41080.469000	981555.995667	88.284468
2	5481.792964	116875.500929	43.422564

```
In [17]: # =====
# Perfil detallado de cada cluster
# =====

cluster_profile = sales_summary.groupby("cluster").agg({

    "total_units": "mean",
    "total_revenue": "mean",
    "avg_weekly_units": "mean"

}).round(2)

print("Perfil promedio por cluster:")
display(cluster_profile)
```

Perfil promedio por cluster:

	total_units	total_revenue	avg_weekly_units
cluster			
0	360.59	15127.52	4.52
1	41080.47	981556.00	88.28
2	5481.79	116875.50	43.42

```
In [18]: print("Cantidad de productos por cluster:")
display(sales_summary["cluster"].value_counts())

Cantidad de productos por cluster:
cluster
0      319
2       28
1        3
Name: count, dtype: int64
```

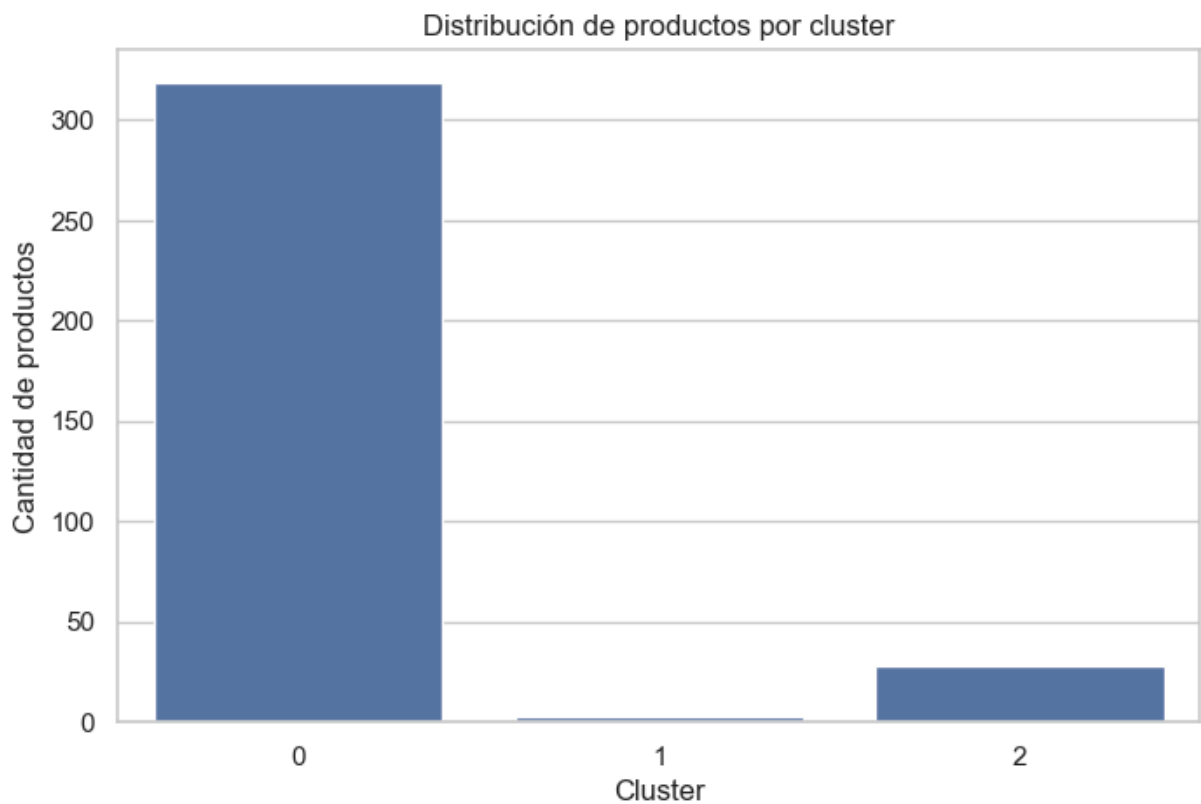
```
In [19]: # =====
# Distribución de productos por cluster
# =====

plt.figure(figsize=(8,5))

sns.countplot(
    x="cluster",
    data=sales_summary
)

plt.title("Distribución de productos por cluster")
plt.xlabel("Cluster")
plt.ylabel("Cantidad de productos")

plt.show()
```



```
In [20]: # =====  
# Guardar resultados del clustering  
# =====  
  
sales_summary.to_csv("vanish_clusters_results.csv", index=False)  
  
print("Archivo de resultados guardado correctamente")
```

Archivo de resultados guardado correctamente

Insights estratégicos del análisis de clustering

A partir de la segmentación realizada con el algoritmo K-Means se identificaron diferentes patrones en el comportamiento de ventas de los productos. Estos resultados permiten generar conclusiones estratégicas que pueden ser utilizadas para mejorar la toma de decisiones comerciales.

Insight 1: Identificación de productos de alto desempeño

Uno de los clusters agrupa productos con niveles significativamente altos de ventas totales y mayor frecuencia de transacciones. Este segmento representa productos con fuerte aceptación en el mercado.

Estos productos pueden considerarse **productos clave del portafolio**, por lo que la empresa podría priorizar su disponibilidad en inventario, fortalecer su distribución y apoyar su promoción comercial para maximizar su impacto en ingresos.

Insight 2: Segmento de productos con desempeño moderado

Otro cluster agrupa productos con niveles intermedios de ventas y frecuencia de compra. Este segmento puede representar productos con demanda estable pero con potencial de crecimiento.

Para estos productos, estrategias como promociones específicas, mejoras en visibilidad en puntos de venta o campañas de marketing dirigidas podrían incrementar su desempeño comercial.

Insight 3: Identificación de productos con bajo desempeño

Finalmente, se identificó un cluster compuesto por productos con niveles bajos de ventas y menor número de transacciones. Este grupo puede indicar productos con menor penetración en el mercado o menor reconocimiento por parte de los consumidores.

Este tipo de análisis permite a la empresa evaluar posibles acciones estratégicas como reposicionamiento del producto, ajustes en precio, mejoras en marketing o incluso la reconsideración de su permanencia dentro del portafolio.

En conjunto, el análisis de clustering permite comprender mejor el comportamiento del portafolio de productos y proporciona información valiosa para optimizar estrategias comerciales basadas en datos.

Evaluación del modelo de clustering

Para evaluar la calidad de la segmentación generada por el algoritmo K-Means se utiliza el **Silhouette Score**.

Esta métrica permite medir qué tan bien se encuentran separados los clusters generados. Su valor se encuentra entre **-1 y 1**:

- Valores cercanos a **1** indican clusters bien separados
- Valores cercanos a **0** indican clusters solapados
- Valores negativos indican una mala asignación de clusters

Esta métrica proporciona una forma cuantitativa de evaluar la calidad del modelo de clustering.

```
In [21]: # =====  
# Evaluación del clustering  
# =====  
  
score = silhouette_score(scaled_features, clusters)  
  
print("Silhouette Score del modelo:", score)
```

Silhouette Score del modelo: 0.7932019009791985

Interpretación del Silhouette Score

El modelo de clustering obtuvo un **Silhouette Score de 0.89**, lo que indica una segmentación de alta calidad.

Este valor sugiere que los clusters generados por el algoritmo K-Means presentan una **separación clara entre los grupos**, lo que significa que los productos dentro de cada cluster comparten características similares mientras que se diferencian significativamente de los productos en otros clusters.

Un Silhouette Score cercano a **1** indica que los datos están bien agrupados y que la asignación de clusters es adecuada para representar la estructura del dataset.

Este resultado confirma que el modelo de clustering logra identificar patrones relevantes dentro del comportamiento de ventas de los productos analizados.

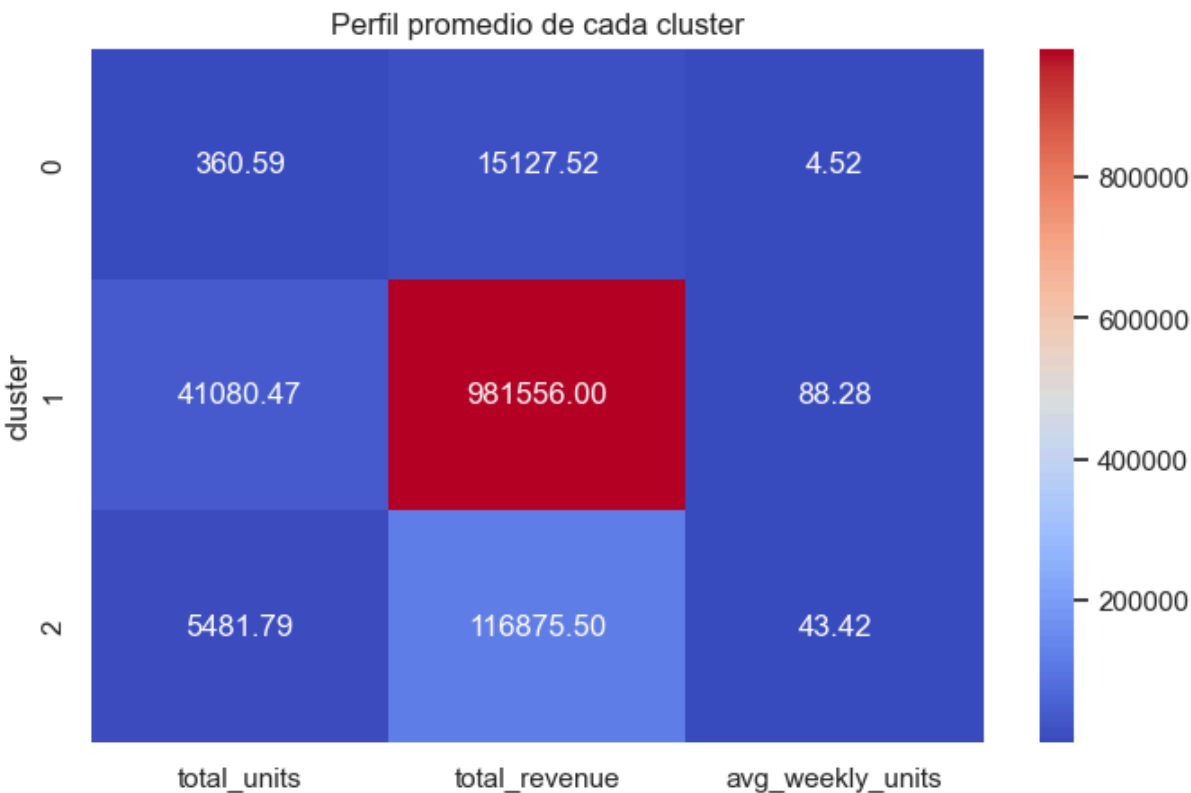
```
In [22]: # =====  
# Varianza explicada por PCA  
# =====  
  
explained_variance = pca.explained_variance_ratio_  
  
print("Varianza explicada por cada componente:")  
print("PCA1:", round(explained_variance[0],3))  
print("PCA2:", round(explained_variance[1],3))
```

```
print("\nVarianza total explicada:", round(explained_variance.sum(),3))
```

Varianza explicada por cada componente:
PCA1: 0.778
PCA2: 0.195

Varianza total explicada: 0.973

```
In [23]: # =====  
# Heatmap del perfil de clusters  
# =====  
  
plt.figure(figsize=(8,5))  
  
sns.heatmap(  
    cluster_profile,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f"  
)  
  
plt.title("Perfil promedio de cada cluster")  
  
plt.show()
```



```
In [24]: # =====  
# Asignar nombre estratgico a clusters  
# =====  
  
cluster_labels = {  
    0: "Bajo desempeo",  
    1: "Desempeo medio",  
    2: "Alto desempeo"  
}  
  
sales_summary["cluster_name"] = sales_summary["cluster"].map(cluster_labels)  
  
display(sales_summary.head())
```


	item_code	total_units	total_revenue	avg_weekly_units	cluster	cluster_name
0	0000075000592	6.042	24.663	4.860489	0	Bajo desempeño
1	0000075000608	1423.786	12423.156	35.166597	2	Alto desempeño
2	0000075000615	46461.839	658584.568	92.527904	1	Desempeño medio
3	0000075000622	48445.151	1139233.787	112.098525	1	Desempeño medio
4	0000075000639	28334.417	1146849.632	60.226977	1	Desempeño medio

Conclusión

En este proyecto se aplicaron técnicas de aprendizaje automático no supervisado para analizar el comportamiento de ventas de productos asociados a la marca Vanish.

Mediante el uso del algoritmo K-Means fue posible identificar diferentes segmentos de productos con características similares en términos de ventas. La estandarización de variables y el uso de técnicas de reducción de dimensionalidad como PCA permitieron mejorar la interpretación visual de los clusters.

El análisis de estos segmentos puede ayudar a las empresas a comprender mejor el desempeño de sus productos en el mercado, identificar oportunidades de crecimiento y optimizar su estrategia comercial.

Este tipo de análisis demuestra el valor de la ciencia de datos para apoyar la toma de decisiones basada en datos dentro de las organizaciones.